Unit Testing

- What is unit testing?
- Why?

Test all you can

We recommend to test each class at a time first as you develop them (Unit Testing). Program parts consisting of several classes can be tested when you are convinced that all classes work fine (Integration Testing).

Successful compilation is only the first step of the process!

Debugging

When testing has made it clear that something is wrong, the cause must be found. The tools that help with this are called Debuggers.

Credits: Wojciech Mostowski, Halmstad University

1.2. How to define a test in JUnit?

A JUnit *test* is a method contained in a class which is only used for testing. This is called a *Test class*. To mark a method as a test method, annotate it with the `@Test` annotation. This method executes the code under test.

The following code defines a minimal test class with one minimal test method.

```
package com.vogella.junit.first;

import static org.junit.jupiter.api.Assertions.assertTrue;

import org.junit.jupiter.api.Test;

class AClassWithOneJUnitTest {

    @Test
    void demoTestMethod() {
        assertTrue(true);
    }
}
```

http://www.vogella.com/tutorials/JUnit/article.html

Example

```
package com.vogella.junit5;

public class Calculator {

    public int multiply(int a, int b) {
        return a * b;
    }
}
```

http://www.vogella.com/tutorials/JUnit/article.html

Example: Test

```java
import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.RepeatedTest;
import org.junit.jupiter.api.Test;

class CalculatorTest {

    Calculator calculator;

    @BeforeEach                                          (1)
    void setUp() {
        calculator = new Calculator();
    }

    @Test                                                (2)
    @DisplayName("Simple multiplication should work")    (3)
    void testMultiply() {
        assertEquals(20, calculator.multiply(4, 5),      (4)
                "Regular multiplication should work");   (5)
    }

    @RepeatedTest(5)                                     (6)
    @DisplayName("Ensure correct handling of zero")
    void testMultiplyWithZero() {
        assertEquals(0, calculator.multiply(0, 5), "Multiple with zero should be zero");
        assertEquals(0, calculator.multiply(5, 0), "Multiple with zero should be zero");
    }
}
```

http://www.vogella.com/tutorials/JUnit/article.html

```java
import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.RepeatedTest;
import org.junit.jupiter.api.Test;

class CalculatorTest {

    Calculator calculator;

    @BeforeEach                                          (1)
```

1. The method annotated with @BeforeEach runs before each test
2. A method annotated with @Test defines a test method
3. @DisplayName can be used to define the name of the test which is displayed to the user
4. This is an assert statement which validates that expected and actual value is the same, if not the message at the end of the method is shown
5. @RepeatedTest defines that this test method will be executed multiple times, in this example 5 times

```java
    @RepeatedTest(5)                                     (6)
    @DisplayName("Ensure correct handling of zero")
    void testMultiplyWithZero() {
        assertEquals(0, calculator.multiply(0, 5), "Multiple with zero should be zero");
        assertEquals(0, calculator.multiply(5, 0), "Multiple with zero should be zero");
    }
}
```

http://www.vogella.com/tutorials/JUnit/article.html

Where to place test files

Typical, unit tests are created in a separate source folder to keep the test code separate from the real code. The standard convention from the Maven and Gradle build tools is to use:

- src/main/java - for Java classes
- src/test/java - for test classes

http://www.vogella.com/tutorials/JUnit/article.html

Testing for exceptions

Testing that certain exceptions are thrown are be done with the `org.junit.jupiter.api.Assertions.expectThrows()` assert statement. You define the expected Exception class and provide code that should throw the exception.

```
import static org.junit.jupiter.api.Assertions.assertThrows;

@Test
void exceptionTesting() {
    // set up user
    Throwable exception = assertThrows(IllegalArgumentException.class, () ->
user.setAge("23"));
    assertEquals("Age must be an Integer.", exception.getMessage());
}
```

http://www.vogella.com/tutorials/JUnit/article.html

AssertALL

If an assert fails in a test, JUnit will stop executing the test and additional asserts are not checked. In case you want to ensure that all asserts are checked you can `assertAll`.

In this grouped assertion all assertions are executed, even after a failure. The error messages get also grouped together.

```
@Test
void groupedAssertions() {
    Address address = new Address();
    assertAll("address name",
        () -> assertEquals("John", address.getFirstName()),
        () -> assertEquals("User", address.getLastName())
    );
}
```

http://www.vogella.com/tutorials/JUnit/article.html

Test Design

What is a good test suite?

- Not necessarily a large one
- A handful of well designed tests is better than a truck load of arbitrary ones
- Ensure coverage of code:
  - Execute every method in each class
  - Execute every reachable line of code in every method
- Automatically generated tests are good if guarantee coverage
- Different coverage requirements may apply

Credits: Wojciech Mostowski, Halmstad University

Tests are Multiplying

Gets difficult…

- Coverage requires lots of tests
- Especially for a big system
- With multiple configurations
- When system changes, tests have to removed, added, or modified

There has to be a better way…

- Model / property based testing
- Test are generated and executed based on a description of the system – its specification consisting of a set of contracts

Credits: Wojciech Mostowski, Halmstad University